
LABORATORY MANUAL FOR MATHEMATICS PRACTICAL (WITH FOSS TOOLS) PYTHON

Discipline Specific Core Course (DSC)

For V Semester
B.Sc. (RNEP)

C5 MAT1P1– Practical on Real Analysis – II



Karnatak University, Dharwad

Effective from
the Academic Year (2026-27)

Prepared by

Dr. Prabha S. Mandalageri

Department of Mathematics

KLEs S.K. Arts College and H.S.K. Science Institute Hubballi

Contents

Lab	List of experiments	Page No.
*	Introduction to Python and installation	1
1	Program to find upper and lower Riemann sums with respect to a given partition	11
2	Program to test Riemann Integrability	12
3	Program to evaluate Riemann integral as a limit of sum	13
4	Program to verify the First Mean Value Theorem of Integral Calculus	14
5	Program to verify the Second Mean Value Theorem of Integral Calculus	15
6	Program to verify the Fundamental Theorem of Calculus with suitable examples	16
7	Program to verify the convergence of an improper integral of first kind	17
8	Program to verify the convergence of an improper integral of second kind	21
9	Program to illustrate Abel's Test with suitable example	25
	Program to illustrate Dirichlet's test with suitable example	27
10	Program to evaluate Beta and Gamma functions for different values	29
11	Program to verify the Beta–Gamma relation with suitable examples	30
12	Program to verify the recurrence relation and duplication formula of the Gamma function with suitable examples	31
**	Summary of Commands	32

PYTHON SOFTWARE:

1. What is Python?

Python is a high-level, general-purpose, open-source programming language. It is widely used in mathematics, scientific computing, data analysis, artificial intelligence, machine learning, engineering and education.

Python has a simple and readable syntax, making it suitable for beginners as well as advanced users.

2. Main Features of Python

- Free and open source
- Easy-to-learn syntax
- Cross-platform – Windows, Linux and macOS
- Interpreted language
- Object-oriented programming support
- Large collection of libraries and packages
- Suitable for mathematical and scientific computing
- Supports interactive programming
- Widely used in Artificial Intelligence and Machine Learning

3. Python Software / Development Environments

Python programs can be written and executed using different software environments.

A. Python IDLE

IDLE stands for Integrated Development and Learning Environment.

It is normally installed along with Python.

It provides:

- Python Shell
- Code editor
- Program execution
- Debugging facilities
- Syntax highlighting

It is suitable for learning basic Python programming.

B. Thonny

Thonny is a simple Python IDE designed especially for beginners.

It provides:

- Simple user interface
- Python editor
- Interactive Shell
- Step-by-step debugging

- Variable inspection
- Easy package management
- Support for Python libraries such as NumPy and SymPy

Thonny is recommended for beginners in mathematical programming.

C. Jupyter Notebook

Jupyter Notebook is an interactive environment in which Python code, mathematical expressions, explanations and output can be written together.

It is particularly useful for mathematics teaching.

For example:

```
import sympy as sp
x = sp.symbols('x')
f = x**2 + 3*x + 2
sp.factor(f)
```

Output:

$(x + 1)(x + 2)$

Jupyter Notebook is useful for demonstrations, lectures and mathematical experiments.

D. Jupyter Lab

JupyterLab is an advanced version of the Jupyter environment.

It provides:

- Multiple notebooks
- Text editors
- Terminals
- File browser
- Interactive output
- Multiple panels and tabs

It is suitable for larger computational projects.

E. Spyder

Spyder is an IDE designed mainly for scientific and engineering programming.

It provides:

- Code editor
- IPython Console
- Variable Explorer
- Debugger
- File browser
- Plotting support

It is useful for NumPy, SciPy, Matplotlib and scientific computing.

F. Anaconda

Anaconda is a Python distribution designed for data science, scientific computing and machine learning.

It provides many packages and tools in one installation.

Common tools include:

- Python
- Jupyter Notebook
- JupyterLab
- Spyder
- NumPy
- SciPy
- Matplotlib
- Pandas

Anaconda is convenient when many scientific packages are required.

G. Pycharm

PyCharm is an **Integrated Development Environment (IDE)** developed by JetBrains. It is specially designed for **Python programming**.

1. What is PyCharm?

PyCharm provides one place to:

- Write Python programs
- Run and test programs
- Find and correct errors
- Install Python packages
- Debug programs
- Manage projects
- Work with virtual environments
- Use libraries such as **SymPy, NumPy, SciPy and Matplotlib**

So, instead of using a simple text editor and Command Prompt separately, PyCharm provides an integrated environment for Python programming.

4. Important Python Packages for Mathematics

Python itself provides the programming language. Additional mathematical functionality is provided through packages.

NumPy

Used for:

- Arrays
- Matrices
- Numerical calculations
- Linear algebra
- Mathematical functions

Example:

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
print(A)
```

SymPy

SymPy is a symbolic mathematics library.

It can be used for:

- Algebra
- Simplification
- Differentiation
- Integration
- Limits
- Series
- Equations
- Matrices

Example:

```
import sympy as sp
x = sp.symbols('x')
print(sp.diff(x**3, x))
print(sp.integrate(x**2, x))
print(sp.limit(sp.sin(x)/x, x, 0))
```

Output:

```
3*x**2
x**3/3
1
```

Matplotlib

Used for graphs and visualization.

Example:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]
plt.plot(x, y)
plt.show()
```

SciPy

SciPy provides advanced scientific and numerical functions.

It is useful for:

- Numerical integration
- Differential equations
- Optimization
- Statistics
- Linear algebra
- Numerical methods

5. Python for Mathematical Computing

Python can be used for both numerical and symbolic mathematics.

Mathematical task	Python package
Numerical calculations	NumPy
Symbolic calculations	SymPy
Graphs	Matplotlib
Numerical integration	SciPy
Differential equations	SciPy
Linear algebra	NumPy/SciPy
Algebra	SymPy
Limits	SymPy
Differentiation	SymPy
Integration	SymPy

6. Basic Python Program

```
a = 10
b = 20
c = a + b
print("Sum =", c)
```

Output:
Sum = 30

7. Python for Mathematics

For example, to calculate $\frac{d}{dx}(x^3 + 2x^2 + 5x)$ using SymPy:

```
import sympy as sp
x = sp.symbols('x')
f = x**3 + 2*x**2 + 5*x
print(sp.diff(f, x))
```

Output:
 $3x^2 + 4x + 5$

Installation of Thonny IDE (Step-by-Step)

Thonny is a free and beginner-friendly Python IDE widely used for learning Python programming.

System Requirements

- Operating System: Windows 10/11, macOS, or Linux
- RAM: Minimum 2 GB (4 GB recommended)
- Internet connection (for downloading)

Step 1: Open the Thonny Website

Open your web browser and visit: <https://thonny.org/>

Step 2: Download Thonny

1. Click the **Download** button.
2. Select the installer for your operating system.
 - Windows (64-bit)
 - macOS
 - Linux

Step 3: Run the Installer

1. Double-click the downloaded installer.
2. Click **Next**.
3. Accept the default installation settings.
4. Click **Install**.
5. Wait for the installation to complete.

Step 4: Launch Thonny

1. Click **Finish**.
2. Open **Thonny** from the Start Menu or Desktop shortcut.

Install SymPy in Thonny — Online

1. Open Thonny.
2. Go to Tools → Manage packages...
3. In the search box, type:
sympy
4. Select SymPy.
5. Click Install.
6. Wait until the installation is completed.
7. Close the package window.

Check whether SymPy is installed

In the Thonny Shell, type:

```
import sympy
```

```
print(sympy.__version__)
```

If a version number appears, for example:

```
1.14.0
```

then SymPy is successfully installed.

Installing Python: Python IDLE

1. Download Python

1. Go to <https://www.python.org/downloads/>
2. Download the latest required Python version.

2. Install Python

1. Double-click the downloaded Python installer.
2. **Important:** Tick “Add python.exe to PATH”.
3. Click “Install Now”.
4. Wait until the installation is completed.

3. Open Python IDLE

1. Click **Start/Search** in Windows.
2. Type **IDLE**.
3. Open **IDLE (Python 3.x)**.
4. You can now start writing and executing Python programs.

Installing python packages

1. Open Command Prompt

1. Click **Start/Search**.
2. Type **cmd**.
3. Open **Command Prompt**.

2. Upgrade pip

Type:
`python -m pip install --upgrade pip`

3. Install NumPy

Type:
`python -m pip install numpy`

4. Install SymPy

Type:
`python -m pip install sympy`

5. Install Matplotlib

Type:
`python -m pip install matplotlib`

6. Install all three packages at once

You can also use:
`python -m pip install numpy sympy matplotlib`

Note: If the above command does not work, use the following command
`py -m pip install sympy`

Check whether the packages are installed

Open Python IDLE and type:

```
import numpy
import sympy
import matplotlib
```

If there is **no error message**, the packages are installed successfully.

You can also check their versions:

```
import numpy
import sympy
import matplotlib
print(numpy.__version__)
print(sympy.__version__)
print(matplotlib.__version__)
```

Installing Anaconda:

1. Download Anaconda

1. Go to the official Anaconda website: <https://www.anaconda.com/download>
2. Select **Windows**.
3. Download the **Windows 64-bit Graphical Installer**.
4. Save the installer file on your computer.

2. Install Anaconda

1. Double-click the downloaded installer.
2. Follow the installation instructions.
3. Select **Just Me** unless administrator installation is required.
4. Complete the installation.

3. Start Anaconda Applications

After installation, open **Anaconda Navigator** from the Start menu.

You can use:

- **JupyterLab** – for interactive Python programming
- **Jupyter Notebook** – for writing and executing Python programs step by step
- **Spyder** – for writing and running Python programs using an IDE

4. Check Python

Open **Anaconda Prompt** and type:

```
python --version
```

You can also check whether packages are available:

```
python -c "import numpy; print(numpy.__version__)"
python -c "import sympy; print(sympy.__version__)"
python -c "import matplotlib; print(matplotlib.__version__)"
```

If no error is displayed, the corresponding package is installed.

Installing Pycharm:

1. Download PyCharm

Go to the official JetBrains website:

<https://www.jetbrains.com/pycharm/download/?section=windows>

Choose **Windows** and download the **Community/free version** if you want a free IDE.

2. Install PyCharm

1. Open the downloaded .exe file.
2. Click **Next**.
3. Select the installation location → **Next**.
4. On the installation options screen, I recommend selecting:
 - ☒ **Create Desktop Shortcut**
 - ☒ **Add "bin" folder to the PATH** (if shown)
 - ☒ **Add "Open Folder as Project" (if shown)**
5. Click **Next** → **Install**.
6. After installation, click **Finish**.

3. Start PyCharm

Open **PyCharm** from the Desktop or Start Menu.

When it starts for the first time, accept the settings/default options.

4. Check Python

Since you are using **SymPy and mathematical programs**, make sure Python is installed.

Open **Command Prompt** and type:

```
python --version
```

You should see something like:

```
Python 3.11.9
```

If Python is not installed, download it from:

[Python official download page](https://www.python.org/downloads/)

<https://www.python.org/downloads/>

During Python installation, **very importantly**, tick:

☒ **Add python.exe to PATH**

5. Create your first PyCharm program

In PyCharm:

New Project → **Pure Python** → **Create**

Then create a Python file:

Right-click project → **New** → **Python File**

Name it:

```
test.py
```

```
print("Hello")
```

Right-click the file → **Run 'test'**.

Step 1: Open PyCharm

Open **PyCharm** → **New Project**.

Choose:

Pure Python

For the Python interpreter, select **New Virtualenv Environment**. This is the easiest and safest option for your projects.

Click **Create**.

Step 2: Install SymPy, NumPy and Matplotlib

In PyCharm, go to:

View → Tool Windows → Terminal

At the bottom, you will see the Terminal.

Type these commands **one by one**:

```
pip install sympy
```

```
pip install numpy
```

```
pip install matplotlib
```

Wait until each installation finishes.

Step 3: Check whether they are installed

Create a Python file called:

```
test.py
```

```
import sympy
```

```
import numpy
```

```
import matplotlib
```

```
print("SymPy version:", sympy.__version__)
```

```
print("NumPy version:", numpy.__version__)
```

```
print("Matplotlib version:", matplotlib.__version__)
```

Click the ► **Run** button.

If the versions are displayed, your installation is successful.

Step 4: Test SymPy

Try this simple program:

```
from sympy import *
```

```
x = symbols('x')
```

```
f = x**2 + 2*x + 1
```

```
print("Function:", f)
```

```
print("Derivative:", diff(f, x))
```

```
print("Integral:", integrate(f, x))
```

Output:

```
Function: x**2 + 2*x + 1
```

```
Derivative: 2*x + 2
```

```
Integral: x**3/3 + x**2 + x
```

Important Note: In PyCharm, if you create a new project, packages installed in one project's virtual environment may not automatically be available in another project. If you get

ModuleNotFoundError: No module named 'sympy', I can show you how to fix it.

Program No 1:

Program to find upper and lower Riemann sums with respect to a given partition

```
from sympy import *
x,i=symbols('x i')
f= sympify(input("Enter the function f(x) : "))
a = sympify(input("Enter lower limit a : "))
b = sympify(input("Enter upper limit b : "))
n = int(input("Enter number of subintervals : "))
h = (b - a) / n
L = summation(f.subs(x, a + (i-1)*h)*h, (i, 1, n))
U = summation(f.subs(x, a + i*h)*h, (i, 1, n))
#L = summation(f.subs(x, a + (i-1)*h), (i, 1, n))*h
#U = summation(f.subs(x, a + i*h), (i, 1, n))*h
print("Lower Riemann Sum =",L)
print("Upper Riemann Sum =",U)
```

Output

```
Enter the function f(x) : x
Enter lower limit a : 0
Enter upper limit b : 1
Enter number of subintervals : 3
Lower Riemann Sum = 1/3
Upper Riemann Sum = 2/3
```

Examples:

1. Let $f(x) = x$ for $x \in [0,1]$. Let P be a partition of $[0,1]$ that divides the interval into three equal subintervals. Find the upper sum $U(P, f)$ and lower sum $L(P, f)$.
2. Let $f(x) = x^2$ for $x \in [0,4]$. Let P be a partition of $[0,4]$ that divides the interval into four equal subintervals. Find the upper sum $U(P, f)$ and lower sum $L(P, f)$.
3. Let $f(x) = x^2$ for $x \in [0,1]$. Let P be a partition of $[0,1]$ that divides the interval into four equal subintervals. Find the upper sum $U(P, f)$ and lower sum $L(P, f)$.
4. Let $f(x) = 2x + 3$ for $x \in [0,1]$. Let P be a partition of $[0,1]$ that divides the interval into four equal subintervals. Find the upper sum $U(P, f)$ and lower sum $L(P, f)$.

Program No.2

Program to test Riemann Integrability

```
from sympy import *
x,i,n=symbols('x i n')
f = sympify(input("Enter the function f(x) : "))
a = sympify(input("Enter lower limit a : "))
b = sympify(input("Enter upper limit b : "))
h = (b - a)/n
L = simplify(summation(f.subs(x, a + (i-1)*h)*h, (i, 1, n)))
U = simplify(summation(f.subs(x, a + i*h)*h, (i, 1, n)))
#L = simplify(summation(f.subs(x, a + (i-1)*h), (i, 1, n))*h)
#U = simplify(summation(f.subs(x, a + i*h), (i, 1, n))*h)
print("Lower Riemann Sum =", L)
print("Upper Riemann Sum =", U)
L1 = simplify(limit(L, n, oo))
U1 = simplify(limit(U, n, oo))
print("Limit of Lower Sum =",L1)
print("Limit of Upper Sum =",U1)
R = integrate(f, (x, a, b))
print("Integral Value =",R)
if L1 ==U1:
    print("f(x) is Riemann Integrable on [a,b]")
else:
    print("f(x) is not Riemann Integrable")
```

Output

```
Enter the function f(x) : x
Enter lower limit a : 0
Enter upper limit b : 1
Lower Riemann Sum = (n - 1)/(2*n)
Upper Riemann Sum = (n + 1)/(2*n)
Limit of Lower Sum = 1/2
Limit of Upper Sum = 1/2
Integral Value = 1/2
f(x) is Riemann Integrable on [a,b]
```

Examples:

1. Show that $f(x) = x$ is R-integrable on $[0,1]$.
2. Show that $f(x) = 2x + 1$ is R-integrable on $[1,2]$.
3. Show that $f(x) = x^2$ is R-integrable on $[0,1]$.
4. Show that $f(x) = 3x + 1$ is R-integrable on $[1,2]$.

Program No.3

Program to evaluate Riemann integral as a limit of sum

```
from sympy import *
x,i,n=symbols('x i n')
f = sympify(input("Enter the function f(x) : "))
a = sympify(input("Enter lower limit a : "))
b = sympify(input("Enter upper limit b : "))
h = (b - a)/n
U = simplify(summation(f.subs(x, a + i*h)*h, (i, 1, n)))
#U = simplify(summation(f.subs(x, a + i*h), (i, 1, n))*h)
print("Riemann Sum =",U)
U1 = simplify(limit(U, n, oo))
print("Riemann Integral as Limit Sum =", U1)
```

Output

```
Enter the function f(x): 3*x+1
Enter lower limit (a): 1
Enter upper limit (b): 2
Riemann Sum = (11*n + 3)/(2*n)
Riemann Integral as Limit Sum = 11/2
```

Examples:

1. Show that $\int_1^2 (3x + 1) dx = \frac{11}{2}$ using definition of integrability
2. Show that $\int_1^2 (2x + 3) dx = 6$ using definition of integrability.
3. Show that $\int_1^2 x dx = \frac{3}{2}$ using definition of integrability.
4. Show that $\int_1^2 (2x + 1) dx = 4$ using definition of integrability.

Program No.4

Program to verify the First Mean Value Theorem of Integral Calculus

```
from sympy import *
x = symbols('x')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
I = N(integrate(f, (x, a, b)))
fc = I/(b-a)
c1 = nsolve(f - fc,float((a+b)/2))
print(f"c={c1:.4f}")
print(f"value of f(c) = {fc:.4f}")
RHS=N(fc*(b-a))
print(f"f(c)(b-a) = { RHS:.4f}")
print(f"Integral = {I:.4f}")
print("First Mean Value Theorem for integrals Verified =",abs(I-RHS) <= 1e-4)
```

First Mean value Theorem of Integral Calculus:

Let $f(x)$ is R-integrable in $[a,b]$ then there exist number $\mu = f(c)$ between the bounds of $f(x)$ such that

$$\int_a^b f(x)dx = f(c) (b - a) \quad c \in [a,b]$$

Output

```
Enter f(x): 2*x+1
Enter lower limit a: 1
Enter upper limit b: 2
c=1.5000
value of f(c) =4.0000
f(c)(b-a) =4.0000
Integral = 4.0000
First Mean Value Theorem for integrals Verified = True
```

Examples:

1. Verify the first mean value theorem of integral calculus for the function $f(x) = 2x + 1$ in $x \in [1,2]$
2. Verify the first mean value theorem of integral calculus for the function $f(x) = 2x + 3$ in $x \in [1,2]$
3. Verify the first mean value theorem of integral calculus for the function $f(x) = 3x + 1$ in $x \in [1,2]$

Program No. 5

Program to verify the Second Mean Value Theorem of Integral Calculus

```
from sympy import *
x, c = symbols('x c')
f = sympify(input("Enter f(x): "))
g = sympify(input("Enter g(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
LHS = N(integrate(f*g,(x,a,b)))
RHS = f.subs(x,a)*integrate(g,(x,a,c))+f.subs(x,b)*integrate(g,(x,c,b))
c1 = nsolve(Eq(LHS, RHS), c, float((a+b)/2))
print(f"LHS = {LHS:.4f}")
print(f"Value of c = {c1:.4f}")
RHS = RHS.subs(c, c1)
print(f"RHS = {RHS:.4f}")
print("Second Mean Value Theorem for integrals Verified =",abs(LHS-RHS) <= 1e-4)
```

Second Mean value Theorem of Integral Calculus:

Let $f(x)$ is R-integrable in $[a,b]$ and $f(x)$ be monotonic and non-negative in $[a,b]$, then for $c \in [a,b]$ such that $\int_a^b f(x)g(x)dx = f(a) \int_a^c g(x)dx + f(b) \int_c^b g(x)dx$

Output

```
Enter f(x): x
Enter g(x): 1
Enter lower limit a: 1
Enter upper limit b: 2
LHS = 1.500
Value of c = 1.500
RHS = 1.500
Second Mean Value Theorem for integrals Verified = True
```

Output

```
Enter f(x): 1/(5+3*cos(x))
Enter g(x): x**2
Enter lower limit a: 0
Enter upper limit b: pi
LHS = 3.7343
Value of c = 2.2550
RHS = 3.7343
Second Mean Value Theorem for integrals Verified = True
```

Examples:

1. Verify the second mean value theorem of integral calculus for $\int_a^b f(x)g(x) dx$, where $f(x) = x, g(x) = 1, [a, b] = [1,2]$.
2. Verify the second mean value theorem of integral calculus for $\int_a^b f(x)g(x) dx$, where $f(x) = \frac{1}{5+3 \cos(x)}, g(x) = x^2, [a, b] = [0, \pi]$.

Program No. 6

Program to verify the Fundamental Theorem of Calculus with suitable examples

```
from sympy import *
x = symbols('x')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
F = integrate(f, x)
LHS = integrate(f, (x, a, b))
RHS = F.subs(x, b) - F.subs(x, a)
print("F(x) =", F)
print("LHS = ", LHS)
print("RHS = ", RHS)
print("Fundamental Theorem of Calculus Verified =", LHS==RHS)
```

Fundamental Theorem of Calculus:

If $F'(x) = f(x)$, then

$$\int_a^b f(x)dx = F(b) - F(a)$$

Or

If $F(x) = \int_a^x f(t)dt$, where f is continuous, then $F'(x) = f(x)$

Output

```
Enter f(x): x**2
Enter lower limit a: 1
Enter upper limit b: 2
F(x) = x**3/3
LHS = 7/3
RHS = 7/3
Fundamental Theorem of Calculus Verified = True
```

Examples:

1. Verify the fundamental theorem of integral calculus for the function $f(x) = x^2$ in $[1,2]$
2. Verify the fundamental theorem of integral calculus for the function $f(x) = x$ in $[0,1]$

Program No. 7

When the improper integral is of the first kind (either a or b are infinite)

$$(i) \int_a^{\infty} f(x)dx = \lim_{t \rightarrow \infty} \int_a^t f(x)dx$$

#Program to verify the convergence of an improper integral of first kind

from sympy import *

x, t = symbols('x t')

f = sympify(input("Enter f(x): "))

a = sympify(input("Enter lower limit a: "))

I = integrate(f, (x, a, t))

R = limit(I, t, oo)

print("Improper Integral =", R)

if R.is_finite:

 print("The Improper Integral is Convergent")

else:

 print("The Improper Integral is Divergent")

Output:

Enter f(x): 1/x**2

Enter lower limit a: 1

Improper Integral = 1

The Improper Integral is Convergent

Output:

Enter f(x): 1/(1+x**2)

Enter lower limit a: 0

Improper Integral = pi/2

The Improper Integral is Convergent

Examples:

1. Test the convergence of the $\int_1^{\infty} \frac{1}{x^2} dx$
2. Test the convergence of the $\int_1^{\infty} \frac{1}{x} dx$
3. Test the convergence of the $\int_0^{\infty} \frac{1}{1+x^2} dx$
4. Test the convergence of the $\int_1^{\infty} \frac{1}{\sqrt{x}} dx$

Program No. 7

$$(ii) \int_{-\infty}^b f(x)dx = \lim_{t \rightarrow -\infty} \int_t^b f(x)dx$$

```
#Program to verify the convergence of an improper integral of first kind
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
b = sympify(input("Enter upper limit b: "))
I = integrate(f, (x, t, b))
R = limit(I, t, -oo)
print("Improper Integral =", R)
if R.is_finite:
    print("The Improper Integral is Convergent")
else:
    print("The Improper Integral is Divergent")
```

Output:

```
Enter f(x): exp(2*x)
Enter upper limit b: 0
Improper Integral = 1/2
The Improper Integral is Convergent
```

Output:

```
Enter f(x): 1/x
Enter upper limit b: 1
Improper Integral = -oo
The Improper Integral is Divergent
```

Examples:

1. Test the convergence of the $\int_{-\infty}^0 x \, dx$
2. Test the convergence of the $\int_{-\infty}^0 e^{2x} \, dx$
3. Test the convergence of the $\int_{-\infty}^0 e^{-x} \, dx$
4. Test the convergence of the $\int_{-\infty}^0 e^x \, dx$
5. Test the convergence of the $\int_{-\infty}^1 \frac{1}{x} \, dx$

Program No. 7

General Program verify the convergence of an improper integral of first kind

#Program to verify the convergence of an improper integral of first kind

```
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
flag=0
if b.is_infinite:
    I = integrate(f, (x, a, t))
    R = limit(I, t, oo)
elif a.is_infinite:
    I = integrate(f, (x, t, b))
    R = limit(I, t, -oo)
else:
    print("improper integral of the first kind is not applicable")
    flag=1
if flag==0:
    print("Improper Integral =", R)
    if R.is_finite:
        print("The Improper Integral is Convergent")
    else:
        print("The Improper Integral is Divergent")
```

When the improper integral is of the first kind (either a or b are infinite)

$$(i) \int_a^{\infty} f(x) dx = \lim_{t \rightarrow \infty} \int_a^t f(x) dx$$

$$(ii) \int_{-\infty}^b f(x) dx = \lim_{t \rightarrow -\infty} \int_t^b f(x) dx$$

1. Test the convergence of the $\int_0^{\infty} \frac{1}{1+x^2} dx$
2. Test the convergence of the $\int_{-\infty}^0 e^{2x} dx$

Output:

```
Enter f(x): 1/(1+x^2)
Enter lower limit a: 0
Enter upper limit b: oo
Improper Integral = pi/2
The Improper Integral is Convergent
```

Output:

```
Enter f(x): exp(2*x)
Enter lower limit a: -oo
Enter upper limit b: 0
Improper Integral = 1/2
The Improper Integral is Convergent
```

Program No. 7

More General Program verify the convergence of an improper integral of first kind

#Program to verify the convergence of an improper integral of first kind

```
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
flag=0
if b.is_infinite:
    I = integrate(f, (x, a, t))
    R = limit(I, t, oo)
elif a.is_infinite:
    I = integrate(f, (x, t, b))
    R = limit(I, t, -oo)
elif a.is_infinite and b.is_infinite:
    I1 = integrate(f, (x, t, c))
    R1 = limit(I1, t, -oo)
    I2 = integrate(f, (x, c, t))
    R2 = limit(I2, t, oo)
    R=simplify(R1+R2)
else:
    print("improper integral of the first kind is not applicable")
    flag=1
if flag==0:
    print("Improper Integral =", R)
    if R.is_finite:
        print("The Improper Integral is Convergent")
    else:
        print("The Improper Integral is Divergent")
```

Output:

```
Enter f(x): 1/(x^2+2*x+2)
Enter lower limit a: -oo
Enter upper limit b: oo
Improper Integral = pi
The Improper Integral is Convergent
```

Output:

```
Enter f(x): 1/(1+x^2)
Enter lower limit a: 0
Enter upper limit b: oo
Improper Integral = pi/2
The Improper Integral is Convergent
```

Output:

```
Enter f(x): exp(2*x)
Enter lower limit a: -oo
Enter upper limit b: 0
Improper Integral = 1/2
The Improper Integral is Convergent
```

When the improper integral is of the first kind (either a or b are infinite)

$$(i) \int_a^\infty f(x)dx = \lim_{t \rightarrow \infty} \int_a^t f(x)dx$$

$$(ii) \int_{-\infty}^b f(x)dx = \lim_{t \rightarrow -\infty} \int_t^b f(x)dx$$

$$(iii) \int_{-\infty}^\infty f(x)dx = \lim_{t_1 \rightarrow -\infty} \int_{t_1}^c f(x)dx$$

$$+ \lim_{t_2 \rightarrow \infty} \int_c^{t_2} f(x)dx$$

1. Test the convergence of the $\int_0^\infty \frac{1}{1+x^2} dx$
2. Test the convergence of the $\int_{-\infty}^0 e^{2x} dx$
3. Test the convergence of the $\int_{-\infty}^\infty \frac{1}{x^2+2x+2} dx$

Program No. 8

When the improper integral is of the second kind ($f(x)$ becomes infinite at either $x=a$ or $x=b$)

(i) If $f(x)$ becomes infinite at $x = a$, $\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_{a+\varepsilon}^b f(x)dx$

Program to test the convergence of an improper integral of second kind

```
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
I = integrate(f, (x, t, b))
R = limit(I, t, a, dir='+')
print("Improper Integral =", R)
if R.is_finite:
    print("The Improper Integral is Convergent")
else:
    print("The Improper Integral is Divergent")
```

Output:

```
Enter f(x): 1/sqrt(x)
Enter lower limit a: 0
Enter upper limit b: 1
Improper Integral = 2
The Improper Integral is Convergent
```

Examples:

1. Test the convergence of the $\int_0^1 \frac{1}{\sqrt{x}} dx$
2. Test the convergence of the $\int_0^1 \log(x) dx$
3. Test the convergence of the $\int_0^1 \frac{1}{x} dx$

(ii) If $f(x)$ becomes infinite at $x = b$, $\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_a^{b-\varepsilon} f(x)dx$

Program to test the convergence of an improper integral of second kind

```
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
I = integrate(f, (x, a, t))
R = limit(I, t, b, dir='-')
print("Improper Integral =", R)
if R.is_finite:
    print("The Improper Integral is Convergent")
else:
    print("The Improper Integral is Divergent")
```

Output:

```
Enter f(x): 1/(sqrt(1-x**2))
Enter lower limit a: 0
Enter upper limit b: 1
Improper Integral = pi/2
The Improper Integral is Convergent
```

Output:

```
Enter f(x): 1/(x**2-1)
Enter lower limit a: 0
Enter upper limit b: 1
Improper Integral = -oo
The Improper Integral is Divergent
```

Examples:

1. Test the convergence of the $\int_0^1 \frac{dx}{\sqrt{1-x^2}}$
2. Test the convergence of the $\int_0^\infty x dx$
3. Test the convergence of the $\int_0^1 \frac{1}{x^2-1} dx$
4. Test the convergence of the $\int_1^4 \frac{4 dx}{x(4-x)}$

Program No. 8

General Program verify the convergence of an improper integral of second kind

Program to verify the convergence of an improper integral of second kind

```
from sympy import *
x, t = symbols('x t')
f = sympify(input("Enter f(x): "))
a = sympify(input("Enter lower limit a: "))
b = sympify(input("Enter upper limit b: "))
flag=0
fa=f.subs(x,a)
fb=f.subs(x,b)
if fa.is_infinite:
# Evaluate integral from t to b
    I = integrate(f, (x, t, b))
# Limit as t → a+
    R = limit(I, t, a, dir='+')
elif fb.is_infinite:
# Evaluate integral from a to t
    I = integrate(f, (x, a, t))
# Limit as t → b-
    R = limit(I, t, b, dir='-')
else:
    print("improper integral of the second kind is not applicable")
    flag=1
if flag==0:
    print("Improper Integral =", R)
    if R.is_finite:
        print("The Improper Integral is Convergent")
    else:
        print("The Improper Integral is Divergent")
```

Output:

```
Enter f(x): 1/sqrt(x)
Enter lower limit a: 0
Enter upper limit b: 1
Improper Integral = 2
The Improper Integral is Convergent
```

Output:

```
Enter f(x): 1/(sqrt(1-x**2))
Enter lower limit a: 0
Enter upper limit b: 1
Improper Integral = pi/2
The Improper Integral is Convergent
```

When the improper integral is of the second kind (f(x) becomes infinite at either x=a or x=b)

(i) If f(x) becomes infinite at x = a

$$\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_{a+\varepsilon}^b f(x)dx$$

(ii) If f(x) becomes infinite at x = b

$$\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_a^{b-\varepsilon} f(x)dx$$

1. Test the convergence of the

$$\int_0^1 \frac{1}{\sqrt{x}} dx$$

2. Test the convergence of the

$$\int_0^1 \frac{dx}{\sqrt{1-x^2}}$$

Program No. 8

More General Program verify the convergence of an improper integral of second kind

Program to verify the convergence of an improper integral of second kind
from sympy import *

Variable

x, t = symbols('x t')

Runtime Input

f = sympify(input("Enter f(x): "))

a = sympify(input("Enter lower limit a: "))

b = sympify(input("Enter upper limit b: "))

flag=0

fa=f.subs(x,a)

fb=f.subs(x,b)

s=singularities(f,x)

c=[p for p in s if p.is_real and a<p<b]

if fa.is_infinite:

Evaluate integral from t to b

I = integrate(f, (x, t, b))

Limit as $t \rightarrow a^+$

R = limit(I, t, a, dir='+')

elif fb.is_infinite:

Evaluate integral from a to t

I = integrate(f, (x, a, t))

Limit as $t \rightarrow b^-$

R = limit(I, t, b, dir='-')

elif len(c)>0:

print("f(x) is infinite at x=:",c[0])

I1 = integrate(f, (x, a, t))

R1 = limit(I1, t, c[0], dir='-')

I2 = integrate(f, (x, t, b))

R2 = limit(I2, t, c[0], dir='+')

R=simplify(R1+R2)

else:

print("improper integral of the second kind is not applicable")

flag=1

if flag==0:

print("Improper Integral =", R)

if R.is_finite:

print("The Improper Integral is Convergent")

else:

print("The Improper Integral is Divergent")

OUTPUT:

Enter f(x): $1/(x^4)^{(1/3)}$

Enter lower limit a: -1

Enter upper limit b: 1

f(x) is infinite at x = 0

Improper Integral = oo

The Improper Integral is Divergent

When the improper integral is of the second kind (f(x) becomes infinite at either $x=a$ or $x=b$)

(i) If f(x) becomes infinite at $x = a$

$$\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_{a+\varepsilon}^b f(x)dx$$

(ii) If f(x) becomes infinite at $x = b$

$$\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_a^{b-\varepsilon} f(x)dx$$

(iii) If f(x) becomes infinite at $x = c$ in $[a, b]$

$$\int_a^b f(x)dx = \lim_{\varepsilon \rightarrow 0} \int_a^{c-\varepsilon} f(x)dx + \lim_{\varepsilon \rightarrow 0} \int_{c+\varepsilon}^b f(x)dx$$

1. Test the convergence of the $\int_{-1}^1 \frac{1}{\sqrt[3]{x^4}} dx$
2. Test the convergence of the $\int_{-1}^1 \frac{dx}{\sqrt[3]{x^2}}$
3. Test the convergence of the $\int_0^1 \frac{1}{\sqrt{x}} dx$
4. Test the convergence of the $\int_0^1 \frac{dx}{\sqrt{1-x^2}}$

OUTPUT:

Enter f(x): $1/(x^2)^{(1/3)}$

Enter lower limit a: -1

Enter upper limit b: 1

f(x) is infinite at x = 0

Improper Integral = 6

The Improper Integral is Convergent

Program No. 9

Abel's Test: If $\int_a^\infty f(x)dx$ is converges and $g(x)$ is bounded and monotonic function for $x \geq a$ then $\int_a^\infty f(x)g(x)dx$ is convergent

```
#Program to illustrate Abel's Test with suitable example
from sympy import *
x = symbols('x')
f = sympify(input("Enter f(x): "))
g = sympify(input("Enter g(x): "))
a = sympify(input("Enter lower limit a: "))
T = oo
I = N(integrate(f, (x, a, T)))
Lg = N(limit(g, x, oo))
print(f"Integral of f(x) = {I:.4f}")
print(f"lim g(x) = {Lg:.2f}")
if I.is_finite and Lg.is_finite:
    print("Integral of f(x) is converges and g(x) is bounded & monotonic")
    print("By Abel's Test, Integral f(x)g(x) is convergent")
else:
    print("By Abel's Test, Integral f(x)g(x) is not convergent")
```

Output:

```
Enter f(x): sin(x)/x
Enter g(x): exp(-x)
Enter lower limit a: 0
Integral of f(x) =1.5708
lim g(x) = 0.00
Integral of f(x) is converges and g(x) is bounded & monotonic
By Abel's Test, Integral f(x)g(x) is convergent
```

Examples:

1. Test the convergence of integral $\int_1^\infty f(x) g(x) dx$, where $f(x) = \frac{\cos(x)}{x^2}$, $g(x) = (1 - e^{-x})$
2. Test the convergence of integral $\int_0^\infty f(x)g(x) dx$, where $f(x) = \frac{\sin(x)}{x}$, $g(x) = e^{-x}$

Abel's Test: If $\int_a^\infty f(x)dx$ is converges and $g(x)$ is bounded and monotonic function for $x \geq a$ then $\int_a^\infty f(x)g(x)dx$ is convergent

#Program to illustrate Abel's Test with suitable example and checking monotonic of $g(x)$

```
from sympy import *
# Variable
x = symbols('x')
# Runtime Input
f = sympify(input("Enter f(x): "))
g = sympify(input("Enter g(x): "))
a = sympify(input("Enter lower limit a: "))
b=oo
dg = diff(g, x)
inc = solveset(dg >= 0, x, domain=Interval(a, b))
dec = solveset(dg <= 0, x, domain=Interval(a, b))
if inc == Interval(a, b):
    print("g(x) is monotonically increasing")
    monotonic = True
elif dec == Interval(a, b):
    print("g(x) is monotonically decreasing")
    monotonic = True
else:
    print("g(x) is not monotonic")
    monotonic = False
# Large value to approximate infinity
T = oo
# Approximate integrals
I = N(integrate(f, (x, a, T)))
# Limit of g(x)
Lg = N(limit(g, x, oo))
print(f"Integral of f(x) = {I:.4f}")
print(f"lim g(x) = {Lg:.2f}")
# Verification
if I.is_finite and Lg.is_finite and monotonic:
    print("Integral of f(x) is converges and g(x) is bounded & monotonic")
    print("By Abel's Test, Integral f(x)g(x) is convergent")
else:
    print("By Abel's Test, Integral f(x)g(x) is not convergent")
```

Find all values of x in $[a,b]$ for which $g'(x) \geq 0$
Here inc gives Interval(1, oo)

Find all values of x in $[a,b]$ for which $g'(x) \leq 0$
Here Dec gives EmptySet

Important: inc and dec contain the **set of points where the derivative has the required sign**, not simply True or False.

Condition	Meaning
$dg \geq 0$ throughout $[a,b]$	increasing / non-decreasing
$dg \leq 0$ throughout $[a,b]$	decreasing / non-increasing
Neither throughout $[a,b]$	Not monotonic

OUTPUT:

```
Enter f(x): cos(x)/(x^2)
Enter g(x): (1-exp(-x))
Enter lower limit a: 1
g(x) is monotonically increasing
Integral of f(x) =-0.0844
lim g(x) = 1.00
Integral of f(x) is converges and g(x) is bounded & monotonic
By Abel's Test, Integral f(x)g(x) is convergent
```

Dirichlet's Test: If $\int_a^t f(x)dx$ is bounded for all $t \geq a$ and $g(x)$ is bounded & monotonic function for $x \geq a$ and $\lim_{x \rightarrow \infty} g(x) = 0$ then $\int_a^\infty f(x)g(x)dx$ is convergent

```
# Program to illustrate Dirichlet's test with suitable example
from sympy import *
x = symbols('x')
f = sympify(input("Enter f(x): "))
g = sympify(input("Enter g(x): "))
a = sympify(input("Enter lower limit a: "))
T = 10
I = N(integrate(f, (x, a, T)))
Lg = N(limit(g, x, oo))
print(f"Integral of f(x) = {I:.4f}")
print("lim g(x) =", Lg)
if I.is_finite and Lg.is_zero:
    print("Integral f(x) is bounded and g(x) is bounded, monotonic and tends to 0")
    print("By Dirichlet's Test, Integral f(x)g(x) is convergent")
else:
    print("By Dirichlet's Test, Integral f(x)g(x) is not convergent")
```

Output:

```
Enter f(x): sin(x)
Enter g(x): 1/x
Enter lower limit a: 1
Integral of f(x) = 1.3794
lim g(x) = 0
Integral f(x) is bounded and g(x) is bounded, monotonic and tends to 0
By Dirichlet's Test, Integral f(x)g(x) is convergent
```

Examples:

1. Test the convergence of integral $\int_1^\infty f(x)g(x) dx$, where $f(x) = \sin(x)$, $g(x) = \frac{1}{x}$
2. Test the convergence of integral $\int_1^\infty f(x)g(x) dx$, where $f(x) = \cos(x)$, $g(x) = \frac{1}{x}$
3. Test the convergence of integral $\int_0^\infty f(x)g(x) dx$, where $f(x) = e^{-x}$, $g(x) = \frac{1}{1+x}$
4. Test the convergence of integral $\int_1^\infty f(x)g(x) dx$, where $f(x) = \sin(x)$, $g(x) = \frac{1}{x^2}$

Dirichlet's Test: If $\int_a^t f(x)dx$ is bounded for all $t \geq a$ and $g(x)$ is bounded & monotonic function for $x \geq a$ and $\lim_{x \rightarrow \infty} g(x) = 0$ then $\int_a^\infty f(x)g(x)dx$ is convergent

Program to illustrate Dirichlet's test with suitable example and checking monotonic of $g(x)$

```
from sympy import *
# Variables
x = symbols('x')
# Runtime Input
f = sympify(input("Enter f(x): "))
g = sympify(input("Enter g(x): "))
a = sympify(input("Enter lower limit a: "))
b=oo
dg = diff(g, x)
inc = solveset(dg >= 0, x, domain=Interval(a, b))
dec = solveset(dg <= 0, x, domain=Interval(a, b))
if inc == Interval(a, b):
    print("g(x) is monotonically increasing")
    monotonic = True
elif dec == Interval(a, b):
    print("g(x) is monotonically decreasing")
    monotonic = True
else:
    print("g(x) is not monotonic")
    monotonic = False
# Numerical approximation (large value of t)
T = 10
# Integral of f(x)
I = N(integrate(f, (x, a, T)))
# Limit of g(x)
Lg = N(limit(g, x, oo))
print(f"Integral of f(x) = {I:.4f}")
print("lim g(x) =", Lg)
# Verification
if I.is_finite and Lg.is_zero and monotonic:
    print("Integral f(x) is bounded and g(x) is bounded, monotonic and tends to 0")
    print("By Dirichlet's Test, Integral f(x)g(x) is convergent")
else:
    print("By Dirichlet's Test, Integral f(x)g(x) is not convergent")
```

Find all values of x in [a,b] for which $g'(x) \geq 0$

Here inc gives EmptySet

Find all values of x in [a,b] for which $g'(x) \leq 0$

Here Dec gives Interval(1, oo)

Important: inc and dec contain the **set of points where the derivative has the required sign**, not simply True or False.

Condition	Meaning
$dg \geq 0$ throughout [a,b]	increasing / non-decreasing
$dg \leq 0$ throughout [a,b]	decreasing / non-increasing
Neither throughout [a,b]	Not monotonic

Output:

```
Enter f(x): sin(x)
Enter g(x): 1/x
Enter lower limit a: 1
g(x) is monotonically decreasing
Integral of f(x) =1.3794
lim g(x) = 0
Integral f(x) is bounded and g(x) is bounded, monotonic and tends to 0
By Dirichlet's Test, Integral f(x)g(x) is convergent
```

Program No. 10

Program to evaluate Beta and Gamma functions for different values

```
from sympy import *
x = symbols('x')
m = sympify(input("Enter m: "))
n = sympify(input("Enter n: "))
b1 = integrate(x**(m-1)*(1-x)**(n-1), (x, 0, 1))
b2 = expand_func(beta(m, n))
print("beta(m,n) =", b1)
print("using built in function, beta(m,n) =", b2)
n = sympify(input("\nEnter n: "))
g1 = integrate(x**(n-1)*exp(-x), (x, 0, oo))
g2 = gamma(n)
print("gamma(n) =", g1)
print("using built in function, gamma(n) =", g2)
```

Output

```
Enter m: 4
Enter n: 3
beta(m,n) = 1/60
using built in function, beta(m,n) = 1/60
```

```
Enter n: 5
gamma(n) = 24
using built in function, gamma(n) = 24
```

Examples:

1. Evaluate $\beta(4,3)$ and $\Gamma(5)$
2. Evaluate $\beta(5,2)$ and $\Gamma(6)$
3. Evaluate $\beta(2,3)$ and $\Gamma(4)$
4. Evaluate $\beta(3,3)$ and $\Gamma(4)$

Example: Evaluate $\beta(4,3)$ and $\Gamma 5$

$$\beta(m, n) = \int_0^1 x^{m-1} (1-x)^{n-1} dx, \quad m, n > 0$$

$$\beta(4,3) = \int_0^1 x^3 (1-x)^2 dx$$

$$= \int_0^1 x^3 (1 + x^2 - 2x) dx$$

$$= \int_0^1 (x^3 + x^5 - 2x^4) dx$$

$$= \left[\frac{x^4}{4} + \frac{x^6}{6} - \frac{2x^5}{5} \right]_0^1 = \frac{1}{60}$$

OR

$$\beta(m, n) = \frac{(m-1)! (n-1)!}{(m+n-1)!}$$

$$\beta(4,3) = \frac{3! 2!}{6!} = \frac{1}{60}$$

and

$$\Gamma n = \int_0^{\infty} x^{n-1} e^{-x} dx, \quad n > 0$$

$$\Gamma n = (n-1)!$$

$$\Gamma 5 = 4! = 24$$

Program No.11

Program to verify the Beta–Gamma relation with suitable examples

```
from sympy import *
m = sympify(input("Enter m : "))
n = sympify(input("Enter n : "))
LHS = expand_func(beta(m, n))
RHS = gamma(m)*gamma(n)/gamma(m+n)
print("LHS =", LHS)
print("RHS =", RHS)
print("Relation between Beta and Gamma Function verified =", LHS == RHS)
```

Output

Enter m: 4
Enter n: 3
LHS = 1/60
RHS = 1/60
Relation between Beta and Gamma Functions verified =True

Output

Enter m: 3
Enter n: 4
LHS = 1/60
RHS = 1/60
Relation between Beta and Gamma Function verified = True

Examples:

1. Verify the Beta–Gamma relation for $\beta(4,3)$
2. Verify the Beta–Gamma relation for $\beta(5,2)$
3. Verify the Beta–Gamma relation for $\beta(2,3)$
4. Verify the Beta–Gamma relation for $\beta(3,2)$

Example: verify relation between beta and gamma functions

$$\beta(4,3) = \int_0^1 x^3 (1-x)^2 dx$$

$$= \int_0^1 x^3 (1+x^2-2x) dx$$

$$= \int_0^1 (x^3 + x^5 - 2x^4) dx$$

$$= \left[\frac{x^4}{4} + \frac{x^6}{6} - \frac{2x^5}{5} \right]_0^1 = \frac{1}{60}$$

OR $\beta(4,3) = \frac{3! 2!}{6!} = \frac{1}{60}$

and $\frac{\Gamma 4 \Gamma 3}{\Gamma 7} = \frac{3! 2!}{6!} = \frac{1}{60}$

Program No.12

Program to verify the recurrence relation and duplication formula of the Gamma function with suitable examples

Program to verify the recurrence relation of the Gamma function

```
from sympy import *
n = sympify(input("Enter n: "))
LHS = gamma(n + 1)
RHS = n * gamma(n)
print("gamma(n+1) = ",LHS)
print("n*gamma(n) = ",RHS)
print("Gamma recurrence relation verified =", LHS == RHS)
```

#Program to verify duplication formula

```
LHS1 = gamma(n) * gamma(n + Rational(1,2))
RHS1 = sqrt(pi) * gamma(2*n) / 2**(2*n - 1)
print("\nLHS = ",LHS1)
print("RHS = ",RHS1)
print("Duplication formula verified =", LHS1== RHS1)
```

Output

```
Enter n: 1/2
gamma(n+1) = sqrt(pi)/2
n*gamma(n) = sqrt(pi)/2
Gamma recurrence relation verified =True
```

```
LHS = sqrt(pi)
RHS = sqrt(pi)
Duplication formula verified =True
```

Output

```
Enter n: 2
gamma(n+1) = 2
n*gamma(n) = 2
Gamma recurrence relation verified =True
```

```
LHS = 3*sqrt(pi)/4
RHS = 3*sqrt(pi)/4
Gamma recurrence relation verified =True
```

Examples:

1. Verify the recurrence relation of the Gamma function and duplication formula for $n=1/2$
2. Verify the recurrence relation of the Gamma function and duplication formula for $n=2$
3. Verify the recurrence relation of the Gamma function and duplication formula for $n=3/2$
4. Verify the recurrence relation of the Gamma function and duplication formula for $n=3$

Summary of Commands

Syntax & Purpose	Example	Output
<input("prompt <ul="" message")=""> - input() is used to accept input from the user during program execution. - The text inside the quotes is displayed as a prompt. </input("prompt>	<pre>name=input("Enter your name: ") print(name)</pre>	Enter your name: Python Python
<pre>print(object1, object2, ...)</pre> It displays the value of variables, expressions, or text on the screen.	<pre>print("Hello") print(10)</pre>	Hello 10
<pre>print(f"Text {variable:format_specifier}")</pre> - f → Indicates a formatted string (f-string). { } → Inserts the value of a variable into the string. :.2f → Displays the value as a floating-point number with 2 digits after the decimal point.	<pre>x = 3.14159265 print(f"Value of x = {x:.2f}")</pre>	Value of x = 3.14
<pre>Sum(expression, (variable, start, end))</pre> It represents the summation symbol Σ	<pre>from sympy import * i = symbols('i') S = Sum(i, (i, 1, 5)) print(S)</pre>	Sum(i, (i, 1, 5))
<pre>expression.doit()</pre> It evaluates symbolic objects such as Sum, Integral, Derivative, etc.	<pre>from sympy import * i = symbols('i') S = Sum(i, (i, 1, 5)) print(S) print(S.doit())</pre>	Sum(i, (i, 1, 5)) 15
<pre>expand(expression)</pre> It expands products and powers	<pre>from sympy import * x = symbols('x') print(expand((x+2)**2))</pre>	$x^2 + 4x + 4$
<pre>summation(expression, (variable, start, end))</pre> It is a function used to find the symbolic sum of a sequence or series, and it evaluates the sum immediately	<pre>from sympy import * i = symbols('i') S = summation(i, (i, 1, 5)) print(S)</pre>	15

Syntax & Purpose	Example	Output
<code>simplify(expression)</code> It simplifies algebraic expressions	<pre>from sympy import * x = symbols('x') expr = (x**2-1)/(x-1) print(simplify(expr))</pre>	$x + 1$
<code>sympify(object)</code> Converts a Python object (such as a string, integer, float, list, or tuple) into an equivalent SymPy expression.	<pre>from sympy import * x = symbols('x') f = sympify("sin(x) + x**2") a = sympify(5) b = sympify("3/7") c=3/7 print(f, a, b) print(c)</pre>	$x^2 + \sin(x)$ 5 3/7 0.42857142857142855 Meaning: • $f \rightarrow$ symbolic expression: $x^2 + \sin(x)$ • $a \rightarrow$ SymPy integer: 5 • $b \rightarrow$ SymPy rational number: 3/7 (exact) • $c \rightarrow$ Python float: 0.42857142857142855 (approximate)
<code>expression.subs(old_variable, new_value)</code> It replaces a variable in an expression with another value or expression	<pre>from sympy import * x = symbols('x') f = x**2 + 3*x print(f.subs(x, 2))</pre>	10
<code>variable1, variable2, ... = symbols('variable1 variable2 ...')</code> It creates symbolic variables that can be used in algebraic expressions, equations, differentiation, integration, summation, etc.	<pre>from sympy import * x = symbols('x') f = x**2 + 3*x + 2 print(f)</pre>	$x^2 + 3x + 2$
	<pre>from sympy import * x, y = symbols('x y') expr = x**2 + y**2 print(expr)</pre>	$x^2 + y^2$
<code>limit(expression, variable, value)</code> It finds the limit of an expression with respect to a variable as x tends to value	<pre>from sympy import * x = symbols('x') print(limit(1/x, x, oo))</pre>	0
<code>limit(expression, variable, value, dir='+')</code> dir $\rightarrow$ Direction of approach. '+' $\rightarrow$ Right-hand limit ($x \rightarrow a^+$) '-' $\rightarrow$ Left-hand limit ($x \rightarrow a^-$) It compute one-sided limits using the dir (direction) argument.	<pre>from sympy import * x = symbols('x') print(limit(1/x, x, 0, dir='+'))</pre>	oo Note: This oo indicates infinity
	<pre>from sympy import * x = symbols('x') print(limit(1/x, x, 0, dir='-'))</pre>	-oo

Syntax & Purpose	Example	Output
<code>integrate(expression, (variable, lower_limit, upper_limit))</code> It computes the definite integral of an expression over the given interval	<pre>from sympy import * x = symbols('x') R = integrate(x**2, (x, 0, 2)) print(R)</pre>	8/3
try: # Code that may produce an error except: # Code to execute if an error occurs - try is used to execute code that may cause an error. - except catches the error and prevents the program from stopping	<pre>try: a = 10 b = 0 print(a/b) except: print("Division by zero is not possible.")</pre>	Division by zero is not possible.
<code>N(expression)</code> Or <code>expression.evalf()</code> It converts an exact symbolic expression into a numerical (decimal) value	<pre>from sympy import * x = pi print(x) print(N(x))</pre>	pi 3.14159265358979
	<pre>from sympy import * x = pi print(x) print(x.evalf())</pre>	pi 3.14159265358979
<code>abs(number)</code> It returns the absolute value (magnitude) of number	<pre>print(abs(10)) print(abs(-10)) print(abs(0))</pre>	10 10 0

Syntax & Purpose	Example	Output
<code>nsolve(equation, variable, initial_guess)</code> The nsolve() function in SymPy is used to find a numerical solution (approximate solution) of an equation or a system of equations The <code>nsolve()</code> does not automatically search for all roots; the initial guess guides it toward a numerical root. So, the initial guess is important because <code>nsolve()</code> is a numerical method and generally finds a root near the supplied starting value.	<pre>from sympy import * x = symbols('x') sol = nsolve(x**2 - 4, x, 1) print(sol)</pre>	2.0000000000000000 Here, $x^2-4=0$ has two solutions, $x=2$ and $x=-2$. Since the initial guess is 1, <code>nsolve()</code> finds the solution near 1, namely 2
<code>expression.is_finite</code> In SymPy, <code>.is_finite</code> is a property used to check whether an expression is finite (not $+\infty$, $-\infty$, or complex infinity) It returns: - True $\rightarrow$ expression is known to be finite - False $\rightarrow$ expression is known to be infinite - None $\rightarrow$ SymPy cannot determine it from the available assumptions	<pre>from sympy import * x = symbols('x') print(sympify(5).is_finite) print(oo.is_finite) print(x.is_finite)</pre> Note: 1) 5 is a Python integer (int), not a SymPy object. The property <code>.is_finite</code> belongs to SymPy objects. Use <code>sympify(5).is_finite</code> or <code>Integer(5).is_finite</code> 2) It is a SymPy property, not a general Python property.	True False None
<code>expression.is_infinite</code> In SymPy, <code>.is_infinite</code> is a property used to check whether an expression is infinite (∞ or $-\infty$) It returns: - True $\rightarrow$ expression is known to be infinite - False $\rightarrow$ expression is known to be finite - None $\rightarrow$ SymPy cannot determine it from the available assumptions	<pre>from sympy import * x = symbols('x') print(sympify(5).is_infinite) print(sympify(oo).is_infinite) print(x.is_infinite)</pre>	False True None

Syntax & Purpose	Example	Output
expression.is_zero In SymPy, .is_zero is a property used to check whether an expression is exactly zero. It returns: • True → expression is definitely zero • False → expression is definitely not zero • None → SymPy cannot determine whether it is zero	<pre>from sympy import * x = symbols('x') print(sympify(2 - 2).is_zero) print((x - x).is_zero) print(sympify(1).is_zero) print(x.is_zero)</pre> Note: 1) In normal Python, 2 - 2 gives the Python integer 0, and Python integers do not have the SymPy property .is_zero. Use sympify(2-2).is_zero 2) It is a SymPy property, not a general Python property.	True True False None
\ In Python, \ is called the backslash. It is commonly used for line continuation or as an escape character. (\ is used to break a long statement into multiple lines.)	<pre>x = 10 + \ 20 print(x)</pre> Here \ tells Python that the statement continues on the next line.	30
	<pre>result = 10 + 20 + 30 + \ 40 + 50 print(result)</pre> Python treats it as one statement	150
singularities(f, x) means “find all points where the function f(x) is singular/not defined with respect to x.” Here: • f → the function in which we want to find singularities. • x → the variable of the function. • singularities(f, x) → finds the singular points of f with respect to x. A singularity is a point where a function is not defined or is not finite. For example, $f(x) = \frac{1}{x - 2}$ At x=2, the denominator becomes zero. therefore, x=2 is a singularity.	<pre>from sympy import * x = symbols('x') f = 1/(x-2) c = singularities(f, x) print(c)</pre>	{2}
	<pre>from sympy import * x = symbols('x') f = 1/((x-1)*(x-3)) c = singularities(f, x) print(c) #print separately for value in c: print(value)</pre>	{1, 3} 1 3

Syntax & Purpose	Example	Output
<code>solveset(expression, x)</code> Find the set of all values of a variable that satisfy an equation or inequality. Here: • expression $\rightarrow$ equation or inequality to solve • x $\rightarrow$ variable • result $\rightarrow$ a set of solutions Inshort: <code>solveset()</code> = solve and return the solutions as a set For monotonicity program, <code>solveset()</code> is generally the better choice. <code>solve()</code> returns solutions usually as a list. <code>solveset()</code> returns the solution as a set.	<pre>sympy import * x = symbols('x') sol = solveset(x**2 - 4, x) print(sol)</pre>	{-2, 2}
	<pre>sol = solveset(2*x >= 0, x) print(sol)</pre>	Interval(0, oo)
	<pre>dg = 2*x a = 1 b = 4 inc = solveset(dg >= 0, x, domain=Interval(a,b)) print(inc)</pre> Here domain=Interval(a,b) specifically tells SymPy: Find the values of x satisfying $g'(x) \leq 0$, only within [a,b] meaning: Find all x in the interval [a,b] for which $g'(x) \geq 0$ Therefore $g'(x) \geq 0$ throughout [1,4], so g(x) is monotonically increasing on [1,4].	Interval(1, 4)
	<pre>f = 1/x df = diff(f, x) dec = solveset(df <= 0, x, domain=Interval(1, 5)) print(dec)</pre> Here domain=Interval(a,b) specifically tells SymPy: Find the values of x satisfying $f'(x) \leq 0$, only within [a,b] meaning: Find all x in the interval [a,b] for which $f'(x) \leq 0$ Therefore $f'(x) \leq 0$ throughout [1,5], so f(x) is monotonically decreasing on [1,5].	Interval(1, 5)

Syntax & Purpose	Example	Output
<code>diff(expression, variable, order)</code> differentiate the given expression with respect to the given variable . where: expression → mathematical expression to differentiate variable → variable with respect to which differentiation is done order → order of derivative; optional	<code>sympy import *</code> <code>x = symbols('x')</code> <code>f = x**3 + 2*x**2 + 5*x</code> <code>df = diff(f, x)</code> <code>print(df)</code>	$3*x**2 + 4*x + 5$
	<code>df = diff(f, x, 2)</code> → second derivative	
Partial Derivative: A partial derivative is the derivative of a function of two or more variables with respect to one variable, while keeping the other variables constant. <code>diff(f, x, m, y, n)</code> → $\left(\frac{\partial^{m+n} f}{\partial x^m \partial y^n}\right)$ - m times differentiation with respect to x - n times differentiation with respect to y - Total order = m+n	<code>sympy import *</code> <code>x, y = symbols('x y')</code> <code>f = x**2*y + x*y**3</code> <code>print(diff(f, x))</code> <code>print(diff(f, y))</code> Meaning Here first print, partial derivative w.r.t to x and y is treated as a constant. $\left(\frac{\partial f}{\partial x} \text{ or } f_x\right)$ Second print, partial derivative w.r.t to y and x is treated as a constant. $\left(\frac{\partial f}{\partial y} \text{ or } f_y\right)$	$2*x*y + y**3$ $x**2 + 3*x*y**2$
	<code>diff(f,x,y)</code> → $\left(\frac{\partial^2 f}{\partial y \partial x} \text{ or } f_{xy}\right)$ <code>diff(f,y,x)</code> → $\left(\frac{\partial^2 f}{\partial x \partial y} \text{ or } f_{yx}\right)$ <code>diff(f,x,2)</code> or <code>diff(f,x,x)</code> → $\left(\frac{\partial^2 f}{\partial x^2} \text{ or } f_{xx}\right)$ <code>diff(f,y,2)</code> or <code>diff(f,y,y)</code> → $\left(\frac{\partial^2 f}{\partial y^2} \text{ or } f_{yy}\right)$ <code>diff(f, x, x, y)</code> or <code>diff(f, x, 2, y)</code> → $\left(\frac{\partial^3 f}{\partial y \partial x^2}\right)$ <code>diff(f, x, 2, y, 3)</code> → $\left(\frac{\partial^5 f}{\partial x^2 \partial y^3}\right)$	